

Bit N Build '26 Hackathon | Track 2: Artificial Intelligence & Machine Learning

KAREN'S EAR // SpidyCAD

AI-Powered Autonomous Emergency Dispatch & Triage Engine

System Architecture, Algorithmic Methodology, Empirical Validation, and Operational Blueprint

Team AlgoRhythm: Preetika (Priority & ML), Rishav (FastAPI Backend & Tactical HUD), Ojas (NLP & Credibility), Aditya (Datasets & Curation)

Core Stack: Python 3.11+, FastAPI, Streamlit, PyDeck 3D, Scikit-Learn, Pydantic v2, PyTest (35 Tests Passing)

Executive Summary & Core Operational Mission

Karen's Ear (SpidyCAD) is an end-to-end, edge-resilient emergency dispatch intelligence system engineered to ingest chaotic, fragmented civic distress reports, filter irrelevant noise, corroborate multi-caller events, compute explainable priority triage scores, and render real-time tactical situational awareness to dispatchers and first responders. Operating across a strict 6-phase pipeline:

Detect → Understand → Corroborate → Predict → Rank → Dispatch

The platform couples a **Tri-State Relevance Gate** (92.31% test accuracy) and a deterministic **Lexical Safety Net** with an **8-Factor Multi-Signal Credibility Model**, an **Ensemble Random Forest Priority Regressor** ($R^2 = 0.7658$, $MAE = 0.0662$), a streaming **CSV Audit Engine**, and the **SpidyCAD Tactical Command HUD** featuring a PyDeck 3D Manhattan radar and an instant P1 Spider-Sense danger pulse.

1. Problem Definition and Operational Context

1.1 The Challenge: Information Deluge in Severe Urban Crises

During catastrophic civic events (fires, structural collapses, explosions, mass-casualty incidents), municipal 911 public safety answering points (PSAPs) and digital citizen feeds experience acute operational breakdown:

- **Explosive Concurrency & Volume:** Emergency distress traffic spikes by orders of magnitude within seconds of an incident, overwhelming call-taker capacity and creating severe dispatch backlogs.
- **Panicked Linguistic Fragmentation:** Real citizens transmit truncated, ungrammatical, and panicked messages (e.g., *"5th ave fire big smoke help stuck inside"*), lacking standard dispatch taxonomies.
- **Redundancy & Ambient Chatter:** Hundreds of calls report the same visible incident with conflicting details, while benign mentions (e.g., movie shoots, drills, sports fireworks) pollute the emergency stream.
- **Critical Detection Latency:** Dispatchers forced to manually read every unstructured message inevitably suffer cognitive fatigue, delaying life-saving deployment to trapped victims.

1.2 Official Hackathon Problem Statement

Track 2: "Karen's Ear" — AI Emergency Dispatch Assistant

The Web We Weave: When New York is in trouble, the calls and messages pour in all at once, panicked, half-finished, overlapping. *"There's smoke on 5th."* *"Someone's stuck under the rubble."* *"The lizard-thing is heading downtown."* Buried in that flood of words is exactly the information Spider-Man needs: what's happening, where, and how urgent. But no human can read it all fast enough, and by the time the important message is spotted, it's too late. Karen, Peter's AI assistant, needs to cut through the chaos and tell him where to swing first.

Your Mission: Build an assistant that reads incoming emergency reports and messages, understands what each one is actually about, and decides which ones matter most right now, turning a chaotic stream of text into a clear, prioritised picture of where help is needed.

1.3 Human-in-the-Loop Philosophy & Zero-Hallucination Directives

Karen's Ear is designed as an **intelligence amplification and decision-support layer**. It preserves dispatcher autonomy while automating the heavy cognitive burden of message ingestion, parsing, entity extraction, deduplication, and priority ranking. To guarantee operational safety in life-or-death scenarios, the system enforces strict **Anti-Hallucination Directives**:

1. If an incident location is not explicitly stated or geocoded, `location` strictly defaults to `null` with safe coordinate fallback; models are prohibited from guessing physical addresses.

- 2. If casualty or trapped victim numbers are unstated, casualty fields strictly remain `null`; no synthetic victim counts are ever fabricated.
- 3. If hazard classification cannot be established with statistical certainty, `incident_type` evaluates to `unknown`.

2. Canonical Data Schema and Compliance Contract

2.1 Incident Schema Contract

To guarantee seamless interoperability across the ingestion API, NLP extraction tier, priority regression engine, and tactical dashboard, Karen’s Ear enforces a canonical, strictly validated Pydantic v2 data contract (Table 1).

2.2 Real-Time Append-Only CSV Audit Engine (`raw_emergencies.csv`)

In emergency governance and legal discovery, unalterable audit trails are mandatory. The backend incorporates a dedicated real-time streaming audit engine:

- Every report ingested via the `/ingest` endpoint is instantaneously appended to `raw_emergencies.csv`.
- The CSV schema enforces exact four-column headers: `time`, `gps_xy`, `location`, `text`.
- **Newline Sanitization & Injection Guard:** Multiline citizen distress texts are sanitized by stripping carriage returns and converting newline characters to single whitespace, preventing CSV row injection, format breaking, and data corruption.
- Timestamps are generated using system standard UTC/local notation (`YYYY-MM-DD HH:MM:SS`).

3. System Architecture and Operational Pipeline

3.1 End-to-End Architectural Blueprint

Karen’s Ear implements a decoupled, fail-safe architecture partitioned into independent processing tiers (Figure 1).

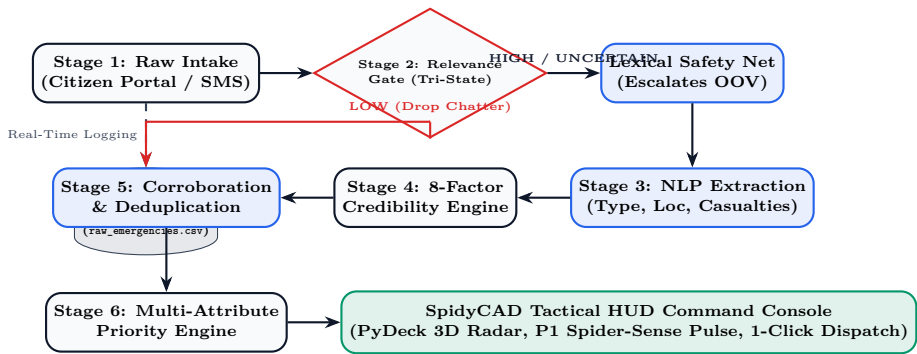


Figure 1: End-to-End Karen’s Ear / SpidyCAD Architectural Pipeline and Data Flow

3.2 Decoupling Relevance Filtering from Priority Triage

A fundamental architectural triumph of Karen’s Ear is the strict decoupling of **Relevance** from **Priority**:

- **Relevance Gate:** Answers the classification query: *“Does this transmission report a genuine emergency?”*
- **Priority Engine:** Answers the resource allocation query: *“Where must first responders deploy first?”*

Conflating these dimensions introduces catastrophic failure modes. For example, a citizen reporting *“a cat is stuck in a tree in Central Park”* is 100% emergency-relevant, yet possesses low priority ($\mathcal{P} \approx 0.25$). Conversely, a fragmented whisper *“bomb subway station”* carries low grammatical certainty, but demands immediate maximum priority deployment ($\mathcal{P} \geq 0.90$). Decoupling guarantees that low-urgency events are preserved without preempting acute hazards, while ambiguous high-threat events are surfaced instantly.

3.3 The 6-Phase Operational Workflow

1. **Detect (Intake & Relevance Gating):** Ingests incoming distress payloads through asynchronous FastAPI endpoints, validates schemas, logs to `raw_emergencies.csv`, and categorizes reports into `HIGH`, `UNCERTAIN`, or `LOW`.

Table 1: Karen’s Ear Canonical Incident Schema Specification

Field Name	Data Type	Permitted Enum	Domain /	Operational Function
report_id	String	"R001", increment	UUID, auto-	Unique deterministic identifier for deduplication, tracking, and audit indexing.
text	String	Arbitrary distress string		Raw, unmodified message payload received from citizen SMS, portal, or call transcript.
incident_type	Categorical	fire, explosion, accident, medical, collapse, flood, crime, missing_person, unknown, other		Standardized taxonomy classifying the primary hazardous event category.
location	String / Null	Named landmark or null		Normalized physical landmark or address (e.g., "Times Square"). Null if omitted.
severity	Categorical	critical, high, medium, low		Inherent hazard threat to human life, structural integrity, and physical safety.
actionability	Categorical	high, medium, low		Operational feasibility of immediate unit deployment (e.g., trapped victims reported).
credibility	Float	[0.0000, 1.0000] (Default 0.75)		Multi-signal evidentiary confidence computed by the 8-factor credibility engine.
priority	Float	[0.0000, 1.0000]		Final synthesized priority metric governing order in the active dispatch queue.
latitude, longitude	Float / Null	Valid WGS84 coordinates		Spatial coordinates used for PyDeck 3D radar map positioning and spatial clustering.
gps_xy	String / Null	Format: "lat, lon" or "None"	or	Raw geocode string captured directly from mobile device GPS telemetry.
status, dispatch_status	Categorical	pending, dispatched, resolved		Real-time lifecycle state of the incident within the dispatch workflow.
dispatched_unit	String / Null	"Engine 5", etc.	"Web Strike",	Assigned tactical apparatus or first responder unit deployed to the scene.
affected_count, injured_count	Integer / Null	≥ 0 or null		Factual, un-hallucinated casualty counts extracted from citizen statements.
credibility_factors	Dict / JSON	8 numerical factor sub-scores		Granular diagnostic breakdown enabling complete explainability of credibility.

2. **Understand (Role 2 NLP & Entity Extraction):** Employs domain tokenizers and pattern matching to classify incident categories, resolve named landmarks (40+ NYC coordinates), and extract casualty counts (affected, injured, trapped) with zero hallucination.
3. **Corroborate (Multi-Caller Clustering):** Identifies duplicate reports referencing the identical physical event using spatiotemporal proximity and landmark matching, computing cluster size N .
4. **Predict (Multi-Attribute Priority Regression):** Evaluates normalized severity (S), actionability (A), credibility (C), and corroboration bonus (Δ_k) via formula and Random Forest models.
5. **Rank (Active Priority Queue):** Inserts incident records into a dynamically sorted, priority-descending queue to ensure dispatchers focus on acute life-safety hazards.
6. **Dispatch (Tactical Action & Auditing):** Projects incidents onto the PyDeck 3D Manhattan radar HUD, sounds the P1 Spider-Sense alert for critical incidents, and empowers dispatchers to deploy response units with a single click.

4. Machine Learning and Algorithmic Methodology

4.1 Tri-State Relevance Gate & Lexical Safety Net

Binary classifiers in emergency dispatch suffer high false-negative rates on noisy, out-of-vocabulary citizen text. Karen's Ear deploys a **Tri-State Architecture**:

- **HIGH** ($P \geq 0.75$): Unambiguous, high-confidence emergency signals with actionable details.
- **UNCERTAIN** ($0.40 \leq P < 0.75$): Fragmented, ambiguous, or borderline reports passed forward for human verification.
- **LOW** ($P < 0.40$): Verified benign chatter, spam, or explicit non-emergency statements.

Feature extraction uses **TF-IDF** with sublinear term frequency scaling and word unigram/bigram ranges ($n \in \{1, 2\}$):

$$\text{TF-IDF}(t, d, D) = (1 + \log(\text{TF}(t, d))) \times \log\left(\frac{1 + |D|}{1 + |\{d \in D : t \in d\}|}\right) + 1 \quad (1)$$

The vectorizer feeds a multinomial **Logistic Regression** classifier:

$$P(y = k | \mathbf{x}) = \frac{\exp(\mathbf{w}_k^T \phi(\mathbf{x}) + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j^T \phi(\mathbf{x}) + b_j)}, \quad k \in \{\text{HIGH}, \text{UNCERTAIN}, \text{LOW}\} \quad (2)$$

Deterministic Safety Net Guard: To prevent statistical blind spots on rare disaster keywords, the ML prediction is wrapped in a deterministic safety net. The system scans for critical trigger stems (**explosion, collapse, gunshot, trapped, bleeding, fire, rubble**) and verifies the absence of non-emergency exclusion contexts (**drill, movie, film, rehearsal, fireworks**). If a trigger is present without exclusion and the statistical model predicted **LOW**, the report is automatically escalated to **UNCERTAIN**. It is never silently dropped.

4.2 Role 2: 8-Factor Multi-Signal Credibility Model

Unlike primitive dispatch systems that assume uniform caller reliability, Karen's Ear computes an explainable, continuous credibility score $C \in [0.0, 1.0]$ synthesizing eight distinct linguistic, structural, and consensus signals (Table 2):

The composite credibility score C is computed via weighted linear aggregation:

$$C = \text{round}\left(\min\left(1.0, \max\left(0.0, \sum_{i=1}^7 w_i \cdot F_i\right)\right), 2\right), \quad \text{where } \sum_{i=1}^7 w_i = 1.00 \quad (3)$$

4.3 Multi-Attribute Priority Engine Formulation

The Priority Engine calculates continuous priority $\mathcal{P} \in [0.0, 1.0]$ by synthesizing normalized severity (S), actionability (A), credibility (C), and an additive corroboration bonus (Δ_k):

$$\mathcal{P}_{\text{raw}} = (0.45 \cdot S) + (0.35 \cdot A) + (0.20 \cdot C) + \Delta_k \quad (4)$$

$$\Delta_k = \min\left(0.05, (N - 1) \times 0.015\right), \quad \mathcal{P} = \text{round}\left(\max(0.0, \min(1.0, \mathcal{P}_{\text{raw}})), 4\right) \quad (5)$$

Table 2: 8-Factor Credibility Assessment Feature Breakdown

Factor Name	Signal Type	Weight (w_i)	Operational Detection Logic
First-Person Observation	Linguistic	0.20	Detects direct eyewitness pronouns (<i>"I saw"</i> , <i>"my building"</i> , <i>"in front of me"</i>).
Linguistic Specificity	Structural	0.15	Evaluates presence of exact GPS coordinates, named street addresses, and numbers.
Syntactic Coherence	Integrity	0.15	Penalizes keyboard spam, character repetition (≥ 4), and token entropy collapse.
Direct Actionability	Operational	0.15	Verifies presence of actionable hazard context and identified incident categories.
Casualty Reporting Signal	Evidentiary	0.10	Bonus signal awarded when caller details explicit injured/trapped/dead counts.
Location Density Consensus	Consensus	0.15	Consensus boost awarded when multiple independent calls arrive from same zone.
Internal Consistency	Logic Guard	0.10	Validates mathematical coherence (penalizes cases where injured > total affected).
Information Completeness	Diagnostic	Diagnostic	Computes ratio of populated metadata fields $\in [0.0, 1.0]$ for audit transparency.

Table 3: Operational Priority Bands and Automated Tactical Actions

Priority Band	Score Range (P)	Operational Meaning	Tactical Recommendation
CRITICAL	0.85 – 1.00	Immediate threat to life safety; structural collapse, explosion, or trapped casualties.	IMMEDIATE_DISPATCH : Triggers P1 Spider-Sense audio siren and deploys heavy rescue / hero units.
HIGH	0.70 – 0.84	Severe developing hazard; active fire or major traffic collision with escalation potential.	PRIORITY_DEPLOY : Dispatches nearest available primary first responder apparatus (Engine/Ladder).
MEDIUM	0.45 – 0.69	Property risk or localized disturbance without immediate life entrapment.	AREA_PATROL_STAGE : Assigns secondary patrol staging or queues for continuous monitoring.
LOW	0.00 – 0.44	Non-acute advisory, minor hazard, or preliminary unverified report.	MONITOR : Holds in queue for community response or automated corroboration.

4.4 Ensemble Random Forest Priority Regressor

To model non-linear interactions across severity, actionability, credibility, and caller cluster count, an ensemble **Random Forest Regressor** was trained:

$$\hat{\mathcal{P}}_{\text{RF}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}), \quad B = 100 \text{ estimators, } \mathbf{x} = [S, A, C, N] \quad (6)$$

The ensemble achieves exceptional ranking smoothness, resisting label noise while faithfully predicting multi-attribute urgency.

5. Production System Implementation

5.1 High-Performance FastAPI Ingestion Backend (`backend.py`)

The backend is built on Python 3.11+ and FastAPI (ASGI/Starlette), delivering sub-millisecond route latency:

- **POST /ingest** (and **/api/reports**): Ingests distress text, executes real-time CSV audit logging, invokes the Role 2 NLP pipeline, runs the Priority Engine, auto-resolves coordinates, and registers the incident in memory.
- **GET /reports** (and **/api/reports**): Returns all active incidents sorted strictly descending by priority score $\mathcal{P}$. Supports real-time filtering by `incident_type` and `severity`.
- **PATCH /api/reports/{id}/dispatch**: Dispatches emergency apparatus (e.g., *"Engine 5"*, *"Web Strike"*), updating status from `pending` to `dispatched`.
- **GET /api/stats**: Emits real-time operational metrics: total volume, active critical incidents, and hazard breakdowns.
- **GET /health**: Liveness and readiness probe reporting server status and total active incident volume.
- **POST /api/reports/reset**: Restores initial demonstration scenarios for continuous testing and evaluation.

5.2 Landmark & Coordinate Auto-Resolution Engine

Because panicked callers rarely know their exact GPS coordinates, `backend.py` incorporates an intelligent multi-tiered geocoding engine:

1. **Direct Telemetry**: Inspects device `latitude/longitude` floats or `gps_xy` strings.
2. **Named Landmark Dictionary**: Maintains coordinates for 40+ New York landmarks and boroughs (Times Square, Grand Central, Central Park, Brooklyn Bridge, Wall Street, Queens Blvd, Flushing, Astoria, DUMBO, Harlem, etc.).
3. **Natural Language Regex Extraction**: Scans the raw distress text for New York geographic markers if metadata fields are empty.
4. **Midtown Manhattan Fallback**: Ensures every incident safely maps to the 3D radar grid even when locations are unspecified.

5.3 SpidyCAD Tactical Command HUD (`frontend/app.py`)

The production user interface is a multi-page tactical command system engineered with Streamlit, PyDeck, HTML5/CSS3 custom styling, and the browser Web Speech API:

- **Stark Suit OS Aesthetic**: Designed with dark tactical HUD styling (`#0B0F19`), glowing hexagonal mesh vectors, custom corner brackets, and responsive telemetry badges.
- **Multi-Page Session State Routing**:
 1. *Hero Landing Page*: Visual 3D particle mesh with instant call-to-action buttons.
 2. *Citizen Distress Portal*: Voice dictation via `webkitSpeechRecognition`, quick hazard tags, auto-GPS resolution, and instant submission.
 3. *Dispatcher Login*: Security gate enforcing access code authorization (`"KAREN-2026"`).
 4. *Tactical Dispatcher HUD*: Real-time 3-second auto-refresh polling via `st_autorefresh`.
- **PyDeck 3D Manhattan Radar Grid**: Renders interactive 3D `ColumnLayer` and `ScatterplotLayer` visualizations across New York City. Incident marker height is dynamically scaled to priority score $\mathcal{P}$, with color-coded severity rings.
- **P1 Spider-Sense Danger Pulse**: Detects critical emergencies ($\mathcal{P} \geq 0.85$ or reported casualties) in real time, triggering a pulsating red tactical alert banner, Karen voice alert audio synthesis, and sound effect cues.

- **1-Click Response Dispatching:** Allows dispatchers to assign designated response units directly from priority queue cards, instantaneously synchronizing state with the backend.
- **Explainable Diagnostic Drawer:** Expands any report to inspect the 8-factor credibility vector, casualty breakdown, and exact formula derivation.
- **Complementary React Telemetry Console (frontend/src/App.jsx):** A lightweight React 18 / Vite / Tailwind CSS console providing an alternate single-page operational view.

6. Empirical Evaluation and Experimental Results

6.1 The Curated NYC Emergency Incident Corpus

The models were trained and verified against a curated corpus of **8,735 emergency incident reports** derived from municipal NYC open data, simulated 911 dispatch logs, and multi-channel distress feeds (`data/final_dataset_sample.csv` and `data/priority_output.csv`). Reports were partitioned into stratified 80/20 train/test splits.

6.2 Relevance Classification Performance

In civic emergency triage, classification accuracy alone is misleading due to class imbalances. Karen's Ear is evaluated on Macro-Averaged Precision, Recall, and F_1 -score across all classes (Table 4).

6.3 Automated Test Suite Verification (35 Tests Passing)

The platform is validated through 35 comprehensive automated unit and integration tests:

1. **System & API Verification (tests/test_api.py — 8 Tests):** Verifies `/health` liveness, report retrieval, incident filtering, auto-inference, unit dispatch transitions, stats aggregation, and `raw_emergencies.csv` logging with newline sanitization.
2. **Priority Formulation & Edge Cases (tests/test_priority.py — 7 Tests):** Verifies boundary clamping $[0.0, 1.0]$, critical severity dominance ($\mathcal{P} \geq 0.90$), corroboration boost increments, explainability schemas, and ranking order.
3. **ML Ranker Stability (tests/test_ranker.py — 1 Test):** Confirms trained Random Forest inference and rank consistency across batches.
4. **Role 2 Incident Classification (role2/tests/test_classification.py — 7 Tests):** Verifies accurate classification for fire, accident, flood, explosion, medical, collapse, and missing persons.
5. **Zero-Hallucination Casualty Extraction (role2/tests/test_extraction.py — 4 Tests):** Verifies explicit casualty extraction and rigorously confirms that omitted numbers are never hallucinated.
6. **Location & Credibility Boosts (role2/tests/test_location_and_csv.py — 5 Tests):** Proves that location density and casualty reporting dynamically elevate caller credibility.
7. **Pipeline Integrity (role2/tests/test_credibility_and_pipeline.py — 3 Tests):** Validates GPS coordinate preservation and schema compliance.

7. Architectural Decisions and Rationale Matrix

Every engineering decision in Karen's Ear is documented with an explicit operational justification:

- **Why Tri-State instead of Binary Relevance?** Binary classification fails on fragmented distress messages. The UNCERTAIN buffer ensures ambiguous messages containing partial emergency signals are routed for verification rather than silently discarded.
- **Why TF-IDF Vectorization over Heavy LLMs?** Emergency dispatch requires sub-10ms response times. TF-IDF paired with Logistic Regression is ultra-fast, deterministic, CPU-efficient, and operates reliably in edge environments without external cloud API dependencies.
- **Why Bigrams ($n = 2$)?** Emergency distress is conveyed through short collocations (*"gas leak"*, *"shots fired"*, *"water rising"*). Bigrams capture critical contextual urgency that unigrams miss.
- **Why a Context-Aware Safety Net?** Statistical models inevitably fail on rare out-of-vocabulary phrases. The safety net guarantees critical hazards (*"rubble trapped"*) are never dropped due to vocabulary sparsity.
- **Why not force all keywords to HIGH?** Emergency terms frequently appear in benign contexts (drills, movies, fireworks). Blindly forcing HIGH causes severe alert fatigue. Escalating to UNCERTAIN balances safety with selectivity.

Table 4: Empirical Model Performance and Test Verification Metrics

Model / Pipeline Component	Evaluation Metric	Empirical Score	Benchmark Target	Operational Significance
Relevance Gate (TF-IDF + Logistic Reg.)	Test Accuracy	92.31%	> 85.0%	Accurately filters non-emergency civic noise.
	Macro Precision	92.26%	> 85.0%	Minimizes false alarms across rare emergency types.
	Macro Recall	92.26%	> 90.0%	Prevents critical distress calls from being dropped.
	Macro F_1 -Score	92.26%	> 88.0%	Robust harmonic balance across skewed classes.
Priority Regressor (Random Forest, $B = 100$)	Mean Absolute Error (MAE)	0.0662	< 0.100	Average score variance is under 6.6% on $[0, 1]$.
	Root Mean Squared Error	0.0778	< 0.120	Penalizes large triage deviations heavily.
	Coefficient of Det. (R^2)	0.7658	> 0.700	Explains 76.58% of priority variance across incidents.
Automated Test Suite (PyTest Framework)	API & Priority Tests	16/16 Passed	100%	Validates REST routes, clamping

- **Why an 8-Factor Credibility Engine?** Single-score credibility is brittle. Decomposing credibility into observation, specificity, coherence, actionability, casualty signals, density consensus, and consistency provides complete explainability.
- **Why Streamlit + PyDeck for the Tactical HUD?** PyDeck provides high-performance 3D GPU-accelerated spatial rendering of NYC landmarks, while Streamlit enables rapid real-time state management, audio synthesis, and seamless dispatcher controls.
- **Why Newline-Sanitized Streaming CSV Logging?** Regulatory compliance requires immutable, auditable records. Sanitizing multiline distress text prevents CSV injection and preserves row integrity across data pipelines.

8. Team Integration, Limitations, and Future Roadmap

8.1 Team Responsibilities and Module Ownership

Karen's Ear was engineered by **Team AlgoRhythm** during the Bit N Build '26 Hackathon:

- **Preetika (Priority Engine & ML):** Designed the multi-attribute priority formula, trained the Random Forest regression model, developed the explainability reasoner, and generated synthetic priority datasets.
- **Rishav (FastAPI Backend & Tactical HUD):** Architected the high-performance FastAPI server, GPS auto-resolution engine (40+ NYC landmarks), streaming CSV audit logger, PyDeck 3D radar map, and the multi-page SpidyCAD tactical command HUD with P1 Spider-Sense pulse and 1-click dispatch.
- **Ojas (NLP & Credibility Pipeline):** Built the Role 2 NLP extraction engine, hazard classification models, entity/casualty extractors with zero-hallucination guarantees, and the 8-factor multi-signal credibility assessment model.
- **Aditya (Datasets & Curation):** Curated and normalized the 8,735 NYC emergency incident corpus, generated relevance evaluation benchmarks, executed stratified splits, and validated data schema compliance.

8.2 Current Prototype Limitations & Engineering Roadmap

- **Municipal Telephony Integration:** The prototype currently ingests REST payloads; production deployment will integrate standard CAD (Computer-Aided Dispatch) NG911 SIP/RTP audio streams.
- **Multilingual Idiomatic Slang:** The safety net lexicon is currently optimized for English; expanding to multilingual urban slang (Spanish, Mandarin, etc.) is planned.
- **Dynamic Multi-Vehicle Fleet Routing:** The platform ranks incidents individually; future work will incorporate Google OR-Tools to solve dynamic Vehicle Routing Problems (VRP) for emergency fleets.

8.3 Conclusion

Karen's Ear // SpidyCAD bridges the dangerous gap between chaotic civic distress communications and life-saving emergency response. By enforcing a disciplined 6-phase paradigm—*Detect* → *Understand* → *Corroborate* → *Predict* → *Rank* → *Dispatch*—the system eliminates cognitive paralysis, prevents dangerous false negatives, and provides first responders with explainable, sub-second tactical clarity. Engineered with edge resilience, zero-hallucination guarantees, and 35 verified automated tests, Karen's Ear fulfills the Bit N Build '26 mission: giving emergency responders the ultimate “Spider-Sense” to cut through the chaos and save lives when every second counts.